SOLO ACCOUNTS

Inventory ของ identity, mailbox, account และ domain พร้อม trust zone และ recovery chain

Product Card

รหัสเอกสาร	CARD-solo-accounts
ฉบับ	0.1
วันที่	2026-10-11
สถานะ	ร่างเพื่อพิจารณา

เอกสารภายในฉบับร่าง — ใช้ประกอบการพัฒนาและดูแลระบบ

สารบัญ

1. สรุปคุณค่าในหนึ่งประโยค
2. ปัญหาและผู้ใช้เป้าหมาย
3. สิ่งที่ทำได้จริงตอนนี้
4. หลักฐานสถานะ
5. คุณค่าและจุดต่าง
6. ช่องว่างสำคัญ
7. คำตัดสิน
8. ขั้นต่อไป
การอนุมัติ
แหล่งข้อมูล

solo-accounts — Product Card

1. สรุปคุณค่าในหนึ่งประโยค

ทะเบียนบัญชีออนไลน์ของเจ้าของ (identity, mailbox, platform account, domain) พร้อม trust zone/MFA/recovery chain โดยไม่เก็บ secret ทำให้ประเมินความเสี่ยงการยึดบัญชีได้จากข้อมูลเดียว

2. ปัญหาและผู้ใช้เป้าหมาย

เจ้าของมีบัญชีหลายชั้น (อีเมลหลายใบ, โดเมน, แพลตฟอร์ม) ที่ผูกกันผ่าน recovery chain การติดตามด้วยความจำทำให้เห็นจุดเสี่ยงไม่ครบ; ผู้ใช้แรกคือเจ้าของ และ README ระบุเป้าหมายเป็น multi-tenant SaaS

3. สิ่งที่ทำได้จริงตอนนี้

- REST `/v1/identities`, `/v1/domains`, `/v1/mailboxes`, `/v1/accounts` (list/stats/get/create/patch/delete)
- ไม่เก็บ password, recovery code, TOTP, cookie, token (ใช้ `vault_ref`) และอีเมลถูกเข้ารหัส AES-256-GCM ฝั่ง web ก่อนส่ง API โดย check constraint ปฏิเสธค่าที่มี `@`
- `trust_zone` หยุดที่ `z2-separated` (persona Z3 ต้องไม่อยู่ใน DB นี้)
- รายงานความเสี่ยง (`tests/test_risk_report.py`) และเว็บ Next.js ใน `web/` สำหรับเจ้าของ (Vercel project `solo-accounts-web` ตาม README)
- ทุกการเขียนผ่านลำดับ precheck → เปลี่ยนข้อมูลในเครื่อง → execution บน empire-platform → commit → finish พร้อม row hash และ RLS แบบบังคับต่อ workspace (ตาม README)

4. หลักฐานสถานะ

รายการ	ค่า
Stack	Python, FastAPI, PostgreSQL (Neon)
Commit	25 ครั้ง (2026-07-27 ถึง 2026-10-11)
ซอร์สโค้ด	51 ไฟล์ / 4777 บรรทัด
เทสต์	8 ไฟล์ (ไม่ได้รันในงานนี้)
CI	ไม่มี workflow
Deploy config	Railway (Serverless) + Neon; ทะเบียน `online-systems.yaml` ระบุสถานะ online (ไม่ได้ probe สดในงานนี้)
Live URL	ดูทะเบียน `online-systems.yaml` (ไม่ได้ probe สด)

5. คุณค่าและจุดต่าง

- เป็นส่วนหนึ่งของสถาปัตยกรรม decentralize ที่ 1 domain = 1 cell จึงเปลี่ยนหรือขายแยกได้
- ใช้สิทธิ์/audit กลางร่วมกันโดยไม่ต้องเขียนใหม่ต่อ cell
- **[สมมติฐาน]** มูลค่าเชิงรายได้ยังไม่พิสูจน์ — ใช้เป็นระบบของเจ้าของก่อน

6. ช่องว่างสำคัญ

- พึ่ง empire-platform ตัวเดียว: ถ้า platform ล่ม cell เขียนไม่ได้ (fail closed)
- working tree มีไฟล์ที่ยังไม่ commit ของงานอื่น (ไม่ได้แตะ)
- ถ้าทำเป็น SaaS ยังไม่มีแผนราคา/ผู้ใช้ภายนอก
- เสียกุญแจ `ACCOUNTS_WEB_FIELD_KEY` = อีเมลที่เก็บอ่านไม่ได้ (ต้องมีสำเนาใน vault)

7. คำตัดสิน

`invest` — ข้อมูลบัญชีเป็นฐานของงาน identity security ที่เจ้าของกำลังทำ และมีการออกแบบความเป็นส่วนตัวที่คิดมาแล้ว จะเปลี่ยนเป็น `showcase` หากเจ้าของยุบ domain นี้หรือย้ายไปรวมกับ cell อื่น

8. ขั้นต่อไป

1. เก็บกุญแจ `ACCOUNTS_WEB_FIELD_KEY` สำรองใน vault และทดสอบการกู้คืน
2. ตัดสินว่าจะทำ multi-tenant SaaS จริงหรือไม่ก่อนลงทุน UI เพิ่ม

การอนุมัติ

บทบาท	สถานะ	วันที่	ลงนาม
Product owner (Book Dev)	รอพิจารณา	—	_

แหล่งข้อมูล

- `README.md`
- `cell.yaml`
- `tests/`